AI Engineering and Claude Architect · Part III, Agents

AI Frameworks, Lesson 3

LlamaIndex and Other Frameworks

A team picked a multi-agent framework from a popular tutorial. A year later they checked its README, which said: "Project archived and no longer maintained." Choosing a framework is partly a bet on who'll still be fixing its security bugs.

Module	AI07 of 13
Lesson	3 of 4
Prerequisites	AI05, AI06 L6, AI07 L1
Exam relevance	Out of scope
Reading time	~25 minutes

Contents

1. LlamaIndex and Other Frameworks

Lesson 3 of 4 · AI Frameworks

LlamaIndex and Other Frameworks

Key idea

LlamaIndex centers on data: loading, chunking, indexing and querying your documents, where LangChain centers on agents. CrewAI centers on teams of agents. **The list keeps changing, and a project's status can change faster than tutorials do, so learn to evaluate a framework by what it hides, who maintains it, and whether you can leave.**

> *Before: RAG (AI05), multi-agent (AI06 L6). Teaches: LlamaIndex's model, what its defaults do, CrewAI, AutoGen's status, and a checklist for evaluating any framework. Used by: AI07 L4.*

The hook: two lines hide a RAG pipeline

This is real LlamaIndex 0.14.25, with its mock LLM and mock embeddings, so no key is needed. Mock embeddings carry no meaning, so **the retrieval order here means nothing.** The point is what it builds, and what it sends. Real output from `code/ai07-frameworks/python/llamaindex_lab.py`:

```
1. One document became 3 nodes
   efdd3c2e metadata={'source': 'refund-policy.md'} text='Card refunds take 5 to 10 business days to appear. Wallet
'
   e3f59e0c metadata={'source': 'refund-policy.md'} text='Bank transfers take up to 14 days because of clearing
rule'
   relationships on node 1: ['SOURCE', 'PREVIOUS', 'NEXT']
2. Two lines of code ran retrieval, built a prompt and called the LLM
3. The prompt LlamaIndex sent for you:
Context information is below.
---------------------
source: refund-policy.md

Bank transfers take up to 14 days because of clearing rules. Orders above $5,000 require approval from a regional
manager.

source: refund-policy.md

Card refunds take 5 to 10 business days to appear. Wallet refunds appear within 24 hours.
---------------------
Given the context information and not prior knowledge, answer the query.
Query: How long do card refunds take?
Answer:
4. Default chunk size: 1024 overlap: 200
```

Key idea

Those two lines made decisions that AI05 spent a module on: a chunk size of 1,024, an overlap of 200, a prompt that doesn't ask for citations, doesn't say what to do when the context is silent, and puts the question last, which is good. **It's a fine start, and it isn't a production design.**

Node IDs vary per run. Also note that it warned me about my tiny `chunk_size=40`, because metadata counts towards chunk size.

Intuition: three specialist shops

Framework	Shop	Sells mainly
LangChain and LangGraph	Hardware store	Agent and workflow parts
LlamaIndex	Library supply	Everything for organizing documents
CrewAI	Staffing agency	Teams of role-playing agents

They overlap, and each now sells a bit of everything. It still helps to ask what each was built for.

Mechanism: LlamaIndex

Its docs call it "a Python toolkit for building LLM-powered agents over your data." It's MIT-licensed. The high-level API takes you from documents to answers in a few lines, while the lower-level API lets you replace any part.

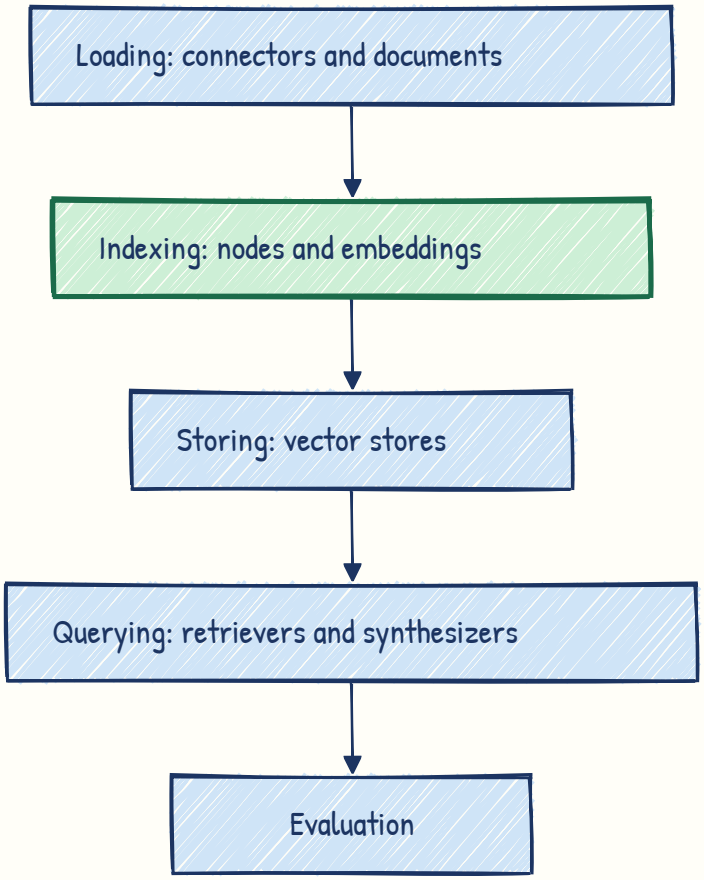


Figure 1. LlamaIndex's five stages, as its docs name them. Each one maps to a topic in AI05, and each default is something you may need to replace.

Concept	What it is	AI05 link
Document	Wraps a data source	Parsing, L2
Node	An atomic chunk, with metadata and links to its source and neighbors	Chunking, L2
Connector (Reader)	Ingests from a source	Connectors, L2
Index	Structure that makes retrieval easy	AI04
Retriever	Fetches context	Search, AI04 L3
Router	Chooses which retriever	Routing, L1 and L5
Node postprocessor	Filters or re-ranks	Rerank, AI04 L3
Response synthesizer	Generates the answer	Context assembly, L4
Workflows	Event-driven steps, with retries and human review	AI06 L1

Notice `PREVIOUS` and `NEXT` in the output. Nodes keep links to neighbors, which supports the parent-child and context expansion ideas from AI05 L2.

LlamaParse is the company's document parsing service, with `LLAMA_CLOUD_API_KEY`. Its quickstart now uses it directly. That's a commercial product alongside the MIT framework, so consider cost and data handling.

What to override for production

Default	Why you might change it
Chunk size 1,024, overlap 200	Fact-integrity testing (AI05 L2)
Vector search only	Hybrid search (AI04 L3)
Prompt template	Citations, abstention, conflicts (AI05 L4)
In-memory index	A real store with permission filters
No permission model of its own in this flow	Filter during search (AI05 L7)

Check the current docs for each before relying on this list. It reflects the query path I ran, and I didn't test the store integrations.

Mechanism: others you'll meet

CrewAI

Its docs describe two pieces, and recommend starting any production app with a Flow:

Piece	Role
Flow	The manager: state, event-driven logic, branches and loops
Crew	A team of role-playing agents with goals and tools, doing a heavy task inside a Flow step

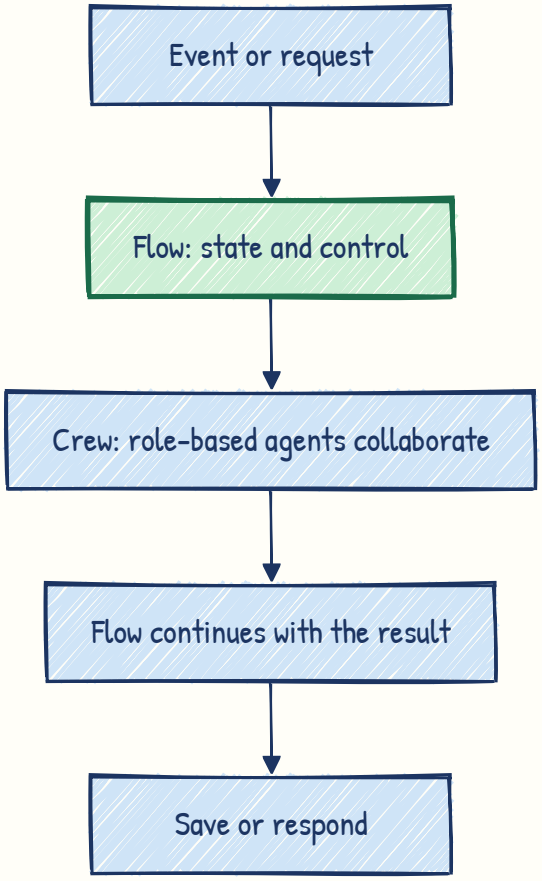


Figure 2. CrewAI's published model. A Flow manages state and decides what's next, and delegates a complex step to a Crew, which returns its result to the Flow.

Note how this echoes AI06 L1: **code-controlled flow by default, with agents only for what needs them.** Role-playing teams are intuitive, but Anthropic's advice still applies: multi-agent costs far more tokens, so check that you need it. The page's "leading framework" wording is its own claim.

AutoGen, and a warning about status

The AutoGen README now says: "Project archived and no longer maintained", and "Microsoft is not providing bug fixes, security fixes, new features, or support for this project." It advises against production use, and it names **Microsoft Agent Framework** as the "enterprise-ready successor".

Yet its documentation site, which I also read, showed no such notice. So a team reading only the docs would never know. **Always check the repository.** I haven't evaluated Microsoft Agent Framework for this course.

Others, unevaluated

You'll also hear of Haystack, Semantic Kernel, DSPy, Pydantic AI and the OpenAI Agents SDK. I name these from general knowledge, and I haven't read their docs for this course, so I'm not describing them. Use the checklist below.

How to evaluate any framework

Question	How to find out	Red flag
Who maintains it, and is it alive?	Repository README, recent releases, open security issues	Archived, or one maintainer
What does it hide?	Log a real request. Read the prompt	You can't see or change prompts
Does it support your model today?	Run it on your actual model	Parameters rejected, or old names only
Provider features?	Try caching, citations, strict tools	Portable layer removes them
What are the runaway limits?	Script a stuck model, as this module does	No limit, or a limit reported as success
How do tool errors reach the model?	Throw an exception containing a secret	Raw messages by default
Can you test without a key?	Write a scripted model	No seam for fakes
Can you leave?	Count framework imports in business code	Everywhere
Where does your data go?	Read about hosted parts and telemetry	Unclear defaults

Scripted-model tests are the cheapest way to find out. Every framework in this module could be tested that way in an afternoon. They revealed things that no feature comparison would.

Why the obvious fixes fall short

Tempting move	Why it disappoints	Better
Compare feature tables	Features are claims, and they go stale	Run the checklist
Choose by GitHub stars	AutoGen had plenty	Check status and releases
Follow a tutorial's choice	It may predate big changes	Check dates
Use a RAG framework's defaults	They aren't tuned for your data	Evaluate (AI05 L6)
Assume a data framework secures data	Permissions are your design	Filter during search

How the field solved it

Need	Common approach
Many data sources	Connector libraries
Tricky documents	Specialized parsers such as LlamaParse
Mixing code and agents	Flows and graphs
Avoiding lock-in	Thin adapters, and MCP for tools

MCP helps here (AI06 L4). Tools behind an MCP server can move between frameworks, which reduces lock-in at that layer.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a framework tutorial in production. It's quick, with default chunking, prompts and in-memory state.

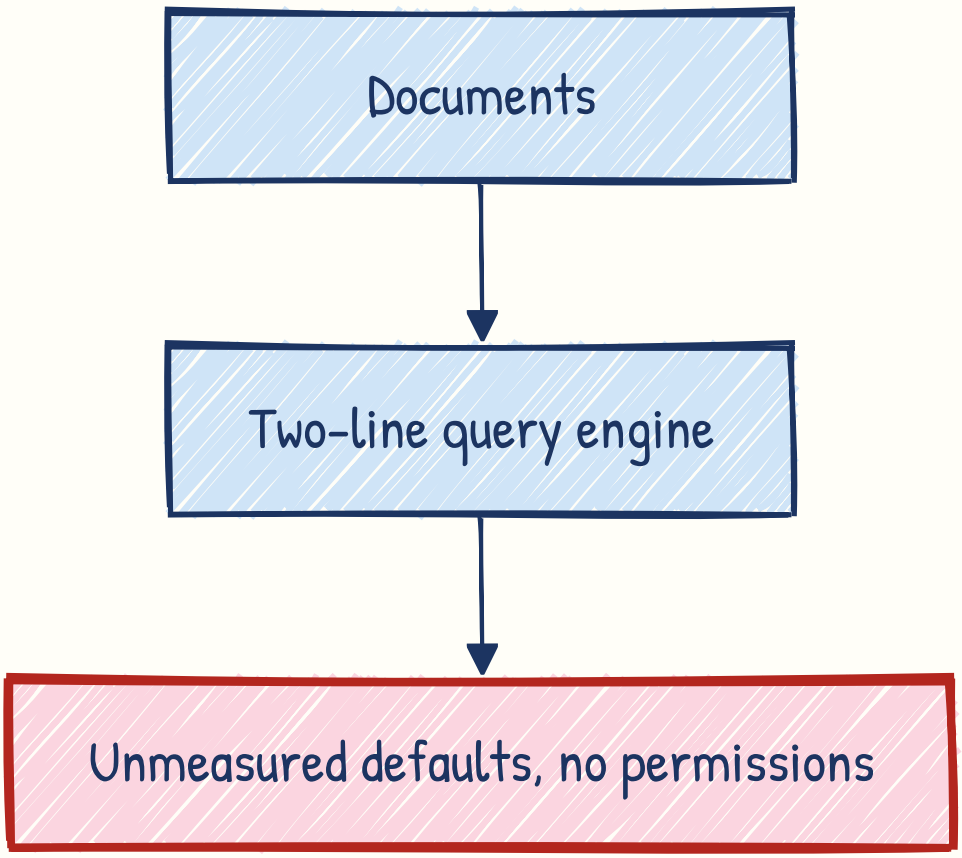


Figure 3. Baseline. Tutorial defaults reach production unchanged and unmeasured.

Improved: the framework with evaluated settings. Quality is measured. New problem: **ingestion, retrieval and prompts are all tied to one framework's types, and to its release schedule.**

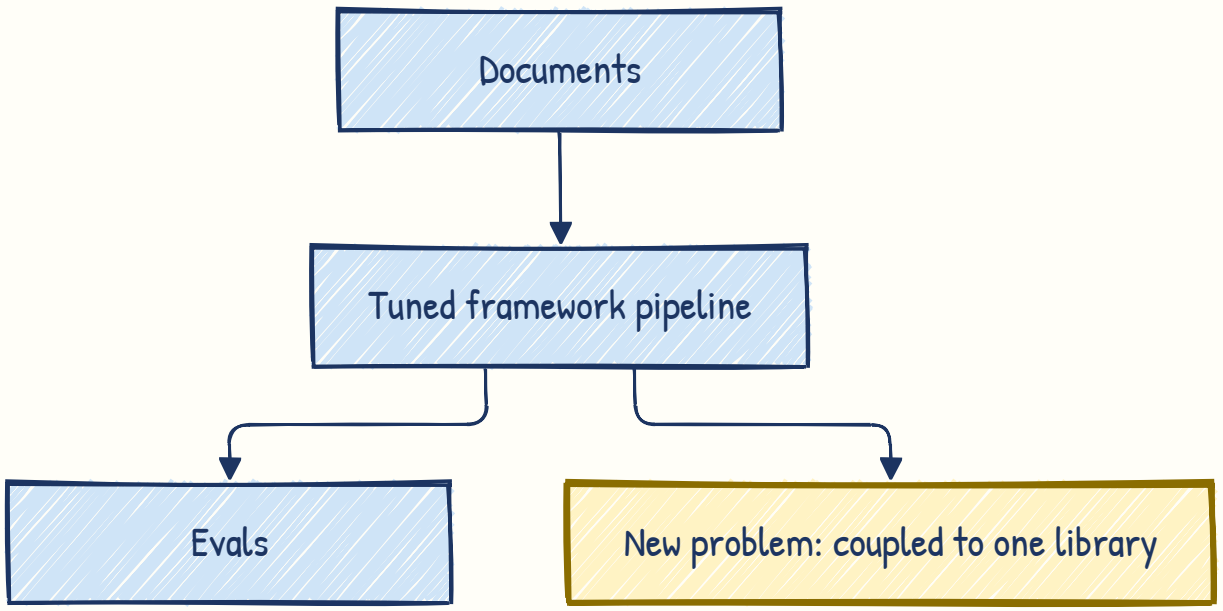


Figure 4. Improved. Settings are tuned against evals, while everything depends on one library's types and releases.

Production: use the framework for what it's good at.

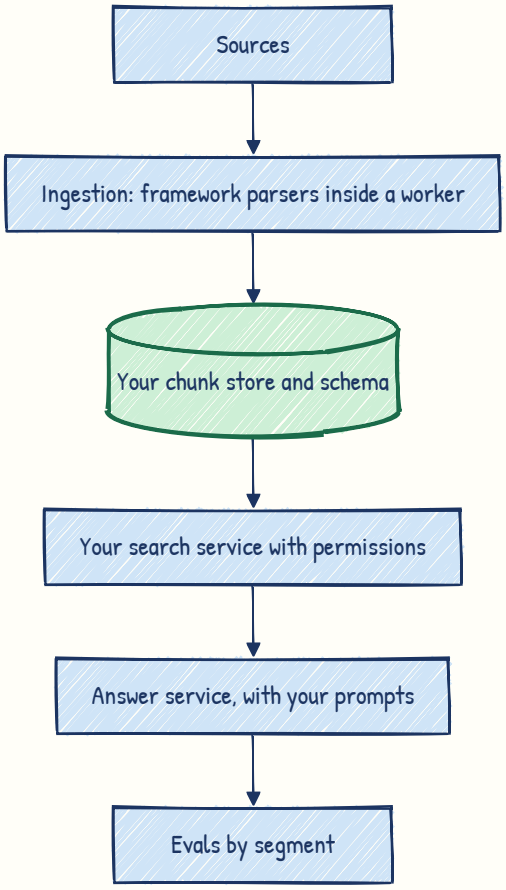


Figure 5. Production. The framework is used inside clear stage boundaries, with chunks stored in your own format. Retrieval and the model call sit behind interfaces you own.

Component	Why it exists	How it communicates
Ingestion worker	Parsers and connectors are the framework's strength	Writes your schema
Chunk store	Your durable format, so you can rebuild without the framework	SQL
Search service	Permissions are yours	Internal API
Answer service	Prompts are yours, in version control	Port (AI07 L1)

Failures

Failure	Containment
Framework archived	Replace the worker, since the data format is yours
Upgrade changes default chunking	Pin versions, and run fact-integrity tests
Hosted parser outage	Queue and retry, with a fallback parser
Prompt template changes on upgrade	Set your own prompts explicitly

Java

These libraries are Python, so there's no Java here. For Java, LangChain4j and Spring AI both offer document loading and RAG (AI07 L4). AI08 L5 builds a RAG service.

Here's the part of the lab that captures the prompt. **This trick works in almost any framework:** subclass the LLM, and read what it receives.

```
class RecordingLLM(MockLLM):
    last_prompt: str = ""

    def complete(self, prompt, formatted=False, **kwargs):
        self.last_prompt = prompt
        return super().complete(prompt, formatted=formatted, **kwargs)
```

Trade-offs

Decision	Gain	Cost
LlamaIndex for RAG	Fast start, many connectors	Defaults to review, and Python
LlamaParse	Handles difficult documents	A paid service, and your data leaves
CrewAI	Intuitive roles and flows	Multi-agent costs, if you lean on crews
Framework-free RAG	Full control	More work

When not to use one: you have one data source, simple needs, and a team able to write 200 lines.

Common mistakes

1. **Trusting two-line demos.** Each line hides real decisions.
2. **Reading only the docs site for status.** AutoGen's archive notice is in the README.
3. **Comparing feature lists.** Run the checklist.
4. **Assuming mock results mean something.** Mock embeddings carry no meaning.
5. **Storing data only in a framework's format.** Keep your own schema.
6. **Assuming permissions are handled.** Check, then design them yourself.

What to say in an interview

"I'd evaluate a framework by checking its status in the repo, logging what it really sends, running it on our model, and scripting a stuck model and a failing tool to see its defaults. AutoGen shows why: the README says it's archived, and its docs site didn't. I'd keep our chunk format and prompts ours, so leaving means replacing a worker and not rebuilding."

Certification lens

How this lesson maps to CCAR-F

Not tested. These are for real projects and interviews.

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Best decision · Medium. A team's RAG prototype uses a two-line query engine, and answers are often wrong. They want to switch frameworks. What do you do first?

A Measure. Read the actual prompt and retrieved nodes, and score retrieval and generation separately (AI05 L6). The defaults, such as a chunk size of 1,024 and vector-only search, may be the cause, and would follow the team to any framework.

Q Q2 · Troubleshooting · Medium. Your company's security review finds that a framework you use is unmaintained, though its docs look current. What's your plan, assuming you followed this lesson's architecture?

A Confirm status in the repository. Assess exposure from its dependencies. Replace it behind its adapter or worker, since data and prompts are in your own formats. Run evals before and after the switch. Without those boundaries, plan a wider migration, and limit risk in the meantime.

Q Q3 · Design · Hard. Write three scripted-model tests you'd run on any new framework, and say what each reveals.

A (1) A model that always calls a tool: reveals the runaway limit, and whether reaching it is reported as an error. (2) A tool that throws an exception containing a secret: reveals what reaches the model. (3) A recording model: reveals actual prompts, tool definitions and history handling. All of them need no key.

Revision - Lesson 3

→ LlamaIndex centers on data: load, index, store, query, evaluate.

→ Documents, nodes, connectors, retrievers, routers, postprocessors, synthesizers.

→ Defaults: chunk size 1,024, overlap 200, a simple context prompt.

→ Override defaults using evals, especially chunking, hybrid search and prompts.

→ CrewAI: Flows for control, Crews for team-of-agents steps.

→ AutoGen's README says it's archived, and names Microsoft Agent Framework as successor.

→ Its docs site showed no such notice, so check the repository.

→ Evaluate: status, what's hidden, model support, limits, errors, testability, exit.

→ Scripted-model tests reveal defaults cheaply.

→ Keep chunk formats and prompts your own.

Your notes